```
!pip install rdkit-pypi torch-geometric

!pip install rdkit
```

```
ERROR: Could not find a version that satisfies the requirement rdkit-pypi (from versions: none)
ERROR: No matching distribution found for rdkit-pypi
Collecting rdkit
  Downloading rdkit-2026.3.1-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (3.8 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rdkit) (2.0.2)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)
Downloading rdkit-2026.3.1-cp312-cp312-manylinux_2_28_x86_64.whl (37.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.0/37.0 MB 60.9 MB/s eta 0:00:00
Installing collected packages: rdkit
Successfully installed rdkit-2026.3.1
```

```
!pip install torch-geometric
```

```
Collecting torch-geometric
  Downloading torch_geometric-2.7.0-py3-none-any.whl.metadata (63 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 63.7/63.7 kB 2.9 MB/s eta 0:00:00
Requirement already satisfied: aiohttp in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.13.4)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (2025.3.0
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (2.0.2)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (5
Requirement already satisfied: pyparsing in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.3.2
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (2.32.4
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from torch-geometric) (3.6.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch-g
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch-geom
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torc
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch-g
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp->torch-
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch-geo
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->torch-geom
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->torc
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->torc
Requirement already satisfied: typing-extensions>=4.2 in /usr/local/lib/python3.12/dist-packages (from aiosignal>
Downloading torch_geometric-2.7.0-py3-none-any.whl (1.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.3/1.3 MB 27.0 MB/s eta 0:00:00
Installing collected packages: torch-geometric
Successfully installed torch-geometric-2.7.0
```

```
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from rdkit import Chem
from rdkit.Chem import rdmolops
from rdkit.Chem import HybridizationType

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch_geometric.data import Data
from torch_geometric.loader import DataLoader
from torch_geometric.nn import GCNConv, global_mean_pool

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, roc_curve
```

```
SEED = 42
BATCH_SIZE = 64
EPOCHS = 100
PATIENCE = 10
LR = 3e-4
WEIGHT_DECAY = 1e-4
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

```
set_seed(SEED)
print("Using device:", DEVICE)
```

```
Using device: cuda
```

```
# Load data
# Replace path if needed
CSV_PATH = "https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv"
df = pd.read_csv(CSV_PATH)

# keep valid rows
df = df[["smiles", "HIV_active"]].dropna().reset_index(drop=True)
df["HIV_active"] = df["HIV_active"].astype(int)
print(df.shape)
df.head()
```

```
(2885, 2)
```

	smiles	HIV_active
0	CNC(=O)Nc1ccncc1	0
1	O=C1NC(O)c2cccc(Cl)c2O1	0
2	O=S1(=O)C=Cc2ccccc2CC1	0
3	Cc1ccc(OS(=O)(=O)c2ccccc2[N+](=O)[O-])cc1	1
4	CN(c1ccccc1)S(=O)(=O)c1cccc([N+](=O)[O-])c1	1

Sonraki adımlar: df ile kod oluşturun | New interactive sheet

```
def atom_features(atom):
    return [
        atom.GetAtomicNum(),
        atom.GetTotalDegree(),
        atom.GetFormalCharge(),
        int(atom.GetHybridization() == HybridizationType.SP),
        int(atom.GetHybridization() == HybridizationType.SP2),
        int(atom.GetHybridization() == HybridizationType.SP3),
        int(atom.GetIsAromatic()),
        int(atom.IsInRing())
    ]

def mol_to_graph(smiles, label):
    mol = Chem.MolFromSmiles(smiles)
    if mol is None:
        return None

    adj = rdmolops.GetAdjacencyMatrix(mol)
    if adj.shape[0] == 0:
        return None

    x = torch.tensor([atom_features(atom) for atom in mol.GetAtoms()], dtype=torch.float)
    edge_index = torch.tensor(np.array(np.nonzero(adj)), dtype=torch.long)
    y = torch.tensor([label], dtype=torch.float)

    return Data(x=x, edge_index=edge_index, y=y)

graphs = []
for _, row in df.iterrows():
    g = mol_to_graph(row["smiles"], row["HIV_active"])
    if g is not None:
        graphs.append(g)

valid_idx = [i for i, row in enumerate(df.itertuples(index=False)) if mol_to_graph(row.smiles, row.HIV_active) is not None]
df = df.iloc[valid_idx].reset_index(drop=True)

print("Valid graphs:", len(graphs))
```

```
Valid graphs: 2885
```

```
class GCNNet(nn.Module):
    def __init__(self, in_channels=8, dropout=0.35):
        super().__init__()
        self.conv1 = GCNConv(in_channels, 128)
        self.bn1 = nn.BatchNorm1d(128)
        self.conv2 = GCNConv(128, 64)
        self.bn2 = nn.BatchNorm1d(64)
        self.conv3 = GCNConv(64, 32)
        self.dropout = nn.Dropout(dropout)
        self.lin1 = nn.Linear(32, 16)
        self.lin2 = nn.Linear(16, 1)

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch
        x = F.relu(self.bn1(self.conv1(x, edge_index)))
```

```
        x = F.relu(self.bn2(self.conv2(x, edge_index)))
        x = F.relu(self.conv3(x, edge_index))
        x = global_mean_pool(x, batch)
        x = self.dropout(x)
        x = F.relu(self.lin1(x))
        x = self.dropout(x)
        x = torch.sigmoid(self.lin2(x)).view(-1)
        return x
```

```
def make_loader(indexes, batch_size=BATCH_SIZE, shuffle=False):
    subset = [graphs[i] for i in indexes]
    return DataLoader(subset, batch_size=batch_size, shuffle=shuffle)

def evaluate(loader, model, loss_fn):
    model.eval()
    total_loss = 0.0
    probs_all, y_all = [], []

    with torch.no_grad():
        for data in loader:
            data = data.to(DEVICE)
            probs = model(data)
            y = data.y.view(-1).float()
            loss = loss_fn(probs, y)
            total_loss += loss.item() * y.size(0)

            probs_all.extend(probs.detach().cpu().numpy())
            y_all.extend(y.detach().cpu().numpy())

    probs_all = np.array(probs_all)
    y_all = np.array(y_all).astype(int)

    # threshold-independent metrics
    auc = roc_auc_score(y_all, probs_all) if len(np.unique(y_all)) > 1 else np.nan
    avg_loss = total_loss / max(len(y_all), 1)
    return avg_loss, y_all, probs_all, auc

def find_best_threshold(y_true, probs):
    thresholds = np.arange(0.10, 0.91, 0.02)
    best_thr, best_f1 = 0.50, -1
    for thr in thresholds:
        pred = (probs >= thr).astype(int)
        score = f1_score(y_true, pred, zero_division=0)
        if score > best_f1:
            best_f1 = score
            best_thr = thr
    return float(best_thr)

def train_one_fold(train_idx, val_idx, test_idx, fold_id):
    set_seed(SEED + fold_id)

    train_loader = make_loader(train_idx, shuffle=True)
    val_loader = make_loader(val_idx, shuffle=False)
    test_loader = make_loader(test_idx, shuffle=False)

    model = GCNNet().to(DEVICE)

    y_train = df.loc[train_idx, "HIV_active"].values
    pos = np.sum(y_train == 1)
    neg = np.sum(y_train == 0)
    pos_weight = torch.tensor([neg / max(pos, 1)], dtype=torch.float, device=DEVICE)

    class WeightedBCELoss(nn.Module):
        def __init__(self, pos_weight):
            super().__init__()
            self.pos_weight = pos_weight
        def forward(self, probs, target):
            eps = 1e-7
            probs = torch.clamp(probs, eps, 1 - eps)
            loss = -(self.pos_weight * target * torch.log(probs) + (1 - target) * torch.log(1 - probs))
            return loss.mean()

    loss_fn = WeightedBCELoss(pos_weight)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR, weight_decay=WEIGHT_DECAY)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="max", factor=0.5, patience=4)

    history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": [], "train_auc": [], "val_auc": []}
    best_state = None
    best_val_auc = -np.inf
    wait = 0

    for epoch in range(1, EPOCHS + 1):
        model.train()
        total_loss = 0.0
        train_probs, train_y = [], []

        for data in train_loader:
            data = data.to(DEVICE)
            optimizer.zero_grad()
            probs = model(data)
            y = data.y.view(-1).float()
```

```
            y = data.y.view(-1).float()
            loss = loss_fn(probs, y)
            loss.backward()
            optimizer.step()

            total_loss += loss.item() * y.size(0)
            train_probs.extend(probs.detach().cpu().numpy())
            train_y.extend(y.detach().cpu().numpy())

        train_probs = np.array(train_probs)
        train_y = np.array(train_y).astype(int)
        train_thr = find_best_threshold(train_y, train_probs)
        train_pred = (train_probs >= train_thr).astype(int)
        train_loss = total_loss / max(len(train_y), 1)
        train_acc = accuracy_score(train_y, train_pred)
        train_auc = roc_auc_score(train_y, train_probs) if len(np.unique(train_y)) > 1 else np.nan

        val_loss, val_y, val_probs, val_auc = evaluate(val_loader, model, loss_fn)
        val_thr = find_best_threshold(val_y, val_probs)
        val_pred = (val_probs >= val_thr).astype(int)
        val_acc = accuracy_score(val_y, val_pred)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["train_acc"].append(train_acc)
        history["val_acc"].append(val_acc)
        history["train_auc"].append(train_auc)
        history["val_auc"].append(val_auc)

        scheduler.step(val_auc)

        if val_auc > best_val_auc + 1e-4:
            best_val_auc = val_auc
            best_state = {k: v.cpu().clone() for k, v in model.state_dict().items()}
            wait = 0
        else:
            wait += 1
            if wait >= PATIENCE:
                break

    model.load_state_dict(best_state)

    _, val_y, val_probs, _ = evaluate(val_loader, model, loss_fn)
    best_threshold = find_best_threshold(val_y, val_probs)

    _, test_y, test_probs, test_auc = evaluate(test_loader, model, loss_fn)
    test_pred = (test_probs >= best_threshold).astype(int)

    fold_result = {
        "fold": fold_id,
        "test_accuracy": accuracy_score(test_y, test_pred),
        "test_precision": precision_score(test_y, test_pred, zero_division=0),
        "test_recall": recall_score(test_y, test_pred, zero_division=0),
        "test_f1": f1_score(test_y, test_pred, zero_division=0),
        "test_roc_auc": test_auc,
        "best_threshold": best_threshold,
        "epochs_ran": len(history["train_loss"])
    }
    return fold_result, history
```

```
all_fold_results = []
all_histories = []

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
indices = np.arange(len(df))
y = df["HIV_active"].values

for fold_id, (train_idx, temp_idx) in enumerate(skf.split(indices, y), start=1):
    temp_y = y[temp_idx]
    val_rel_idx, test_rel_idx = train_test_split(
        np.arange(len(temp_idx)),
        test_size=0.5,
        random_state=SEED + fold_id,
        stratify=temp_y
    )
    val_idx = temp_idx[val_rel_idx]
    test_idx = temp_idx[test_rel_idx]

    print(f"\n===== Fold {fold_id} =====")
    print("Train:", len(train_idx), "Val:", len(val_idx), "Test:", len(test_idx))

    fold_result, hist = train_one_fold(train_idx, val_idx, test_idx, fold_id)
    print(fold_result)

    all_fold_results.append(fold_result)
    all_histories.append(hist)

results_df = pd.DataFrame(all_fold_results)
results_df
```

```
===== Fold 1 =====
Train: 2308 Val: 288 Test: 289
{'fold': 1, 'test_accuracy': 0.698961937716263, 'test_precision': 0.6685393258426966, 'test_recall': 0.8095238095

===== Fold 2 =====
Train: 2308 Val: 288 Test: 289
{'fold': 2, 'test_accuracy': 0.7301038062283737, 'test_precision': 0.7633587786259542, 'test_recall': 0.680272108

===== Fold 3 =====
Train: 2308 Val: 288 Test: 289
{'fold': 3, 'test_accuracy': 0.7301038062283737, 'test_precision': 0.7777777777777778, 'test_recall': 0.662162162

===== Fold 4 =====
Train: 2308 Val: 288 Test: 289
{'fold': 4, 'test_accuracy': 0.726643598615917, 'test_precision': 0.6825396825396826, 'test_recall': 0.8716216216

===== Fold 5 =====
Train: 2308 Val: 288 Test: 289
{'fold': 5, 'test_accuracy': 0.7093425605536332, 'test_precision': 0.6568627450980392, 'test_recall': 0.905405405
```

	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc	best_threshold	epochs_ran
0	1	0.698962	0.668539	0.809524	0.732308	0.797595	0.34	79
1	2	0.730104	0.763359	0.680272	0.719424	0.785810	0.48	58
2	3	0.730104	0.777778	0.662162	0.715328	0.811290	0.58	100
3	4	0.726644	0.682540	0.871622	0.765579	0.837694	0.42	64
4	5	0.709343	0.656863	0.905405	0.761364	0.830698	0.30	61

Sonraki adımlar: `results_df` ile kod oluşturun | New interactive sheet

```
summary_rows = []
metric_cols = ["test_accuracy", "test_precision", "test_recall", "test_f1", "test_roc_auc"]

for col in metric_cols:
    mean = results_df[col].mean()
    std = results_df[col].std(ddof=1)
    var = results_df[col].var(ddof=1)
    summary_rows.append({
        "Metric": col.replace("test_", "").upper(),
        "Mean": mean,
        "Std": std,
        "Variance": var,
        "Formatted": f"{mean:.3f} ± {std:.3f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.719031	0.014140	0.000200	0.719 ± 0.014
1	PRECISION	0.709816	0.056430	0.003184	0.710 ± 0.056
2	RECALL	0.785797	0.110291	0.012164	0.786 ± 0.110
3	F1	0.738801	0.023424	0.000549	0.739 ± 0.023
4	ROC_AUC	0.812617	0.021805	0.000475	0.813 ± 0.022

Sonraki adımlar: `summary_df` ile kod oluşturun | New interactive sheet

```
final_table = pd.DataFrame([{
    "Model": "GCN",
    "Accuracy": summary_df.loc[summary_df["Metric"]=="ACCURACY", "Formatted"].iloc[0],
    "Precision": summary_df.loc[summary_df["Metric"]=="PRECISION", "Formatted"].iloc[0],
    "Recall": summary_df.loc[summary_df["Metric"]=="RECALL", "Formatted"].iloc[0],
    "F1": summary_df.loc[summary_df["Metric"]=="F1", "Formatted"].iloc[0],
    "ROC-AUC": summary_df.loc[summary_df["Metric"]=="ROC_AUC", "Formatted"].iloc[0],
}])
final_table
```

	Model	Accuracy	Precision	Recall	F1	ROC-AUC
0	GCN	0.719 ± 0.014	0.710 ± 0.056	0.786 ± 0.110	0.739 ± 0.023	0.813 ± 0.022

```
results_df.to_csv("GCN_fold_results.csv", index=False)
summary_df.to_csv("GCN_summary_results.csv", index=False)
```

```
final_table.to_csv("GCN_final_table.csv", index=False)
print("Saved: GCN_fold_results.csv, GCN_summary_results.csv, GCN_final_table.csv")
```

```
Saved: GCN_fold_results.csv, GCN_summary_results.csv, GCN_final_table.csv
```

```
# Average training curves across folds
max_len = max(len(h["train_loss"]) for h in all_histories)

def pad_series(series, target_len):
    if len(series) == target_len:
        return np.array(series, dtype=float)
    arr = np.array(series, dtype=float)
    pad_value = arr[-1]
    return np.pad(arr, (0, target_len - len(arr)), constant_values=pad_value)

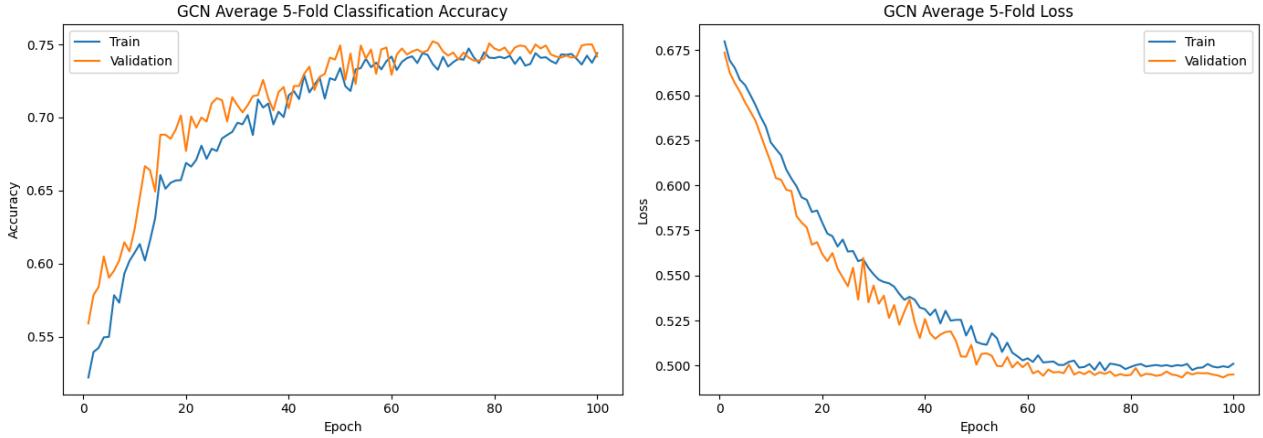
avg_train_acc = np.mean([pad_series(h["train_acc"], max_len) for h in all_histories], axis=0)
avg_val_acc   = np.mean([pad_series(h["val_acc"], max_len) for h in all_histories], axis=0)
avg_train_loss = np.mean([pad_series(h["train_loss"], max_len) for h in all_histories], axis=0)
avg_val_loss   = np.mean([pad_series(h["val_loss"], max_len) for h in all_histories], axis=0)

epochs = np.arange(1, max_len + 1)

plt.figure(figsize=(14,5))
plt.subplot(1,2,1)
plt.plot(epochs, avg_train_acc, label="Train")
plt.plot(epochs, avg_val_acc, label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("GCN Average 5-Fold Classification Accuracy")
plt.legend()

plt.subplot(1,2,2)
plt.plot(epochs, avg_train_loss, label="Train")
plt.plot(epochs, avg_val_loss, label="Validation")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("GCN Average 5-Fold Loss")
plt.legend()

plt.tight_layout()
plt.show()
```




```
# === GCN Average Curves + Early Stopping  ===
import numpy as np
import matplotlib.pyplot as plt


max_len = max(len(h["train_loss"]) for h in all_histories)

def pad_series(series, target_len):
    arr = np.array(series, dtype=float)
    if len(arr) == target_len:
        return arr
    return np.pad(arr, (0, target_len - len(arr)), constant_values=arr[-1])

# Ortalama eğriler
avg_train_acc  = np.mean([pad_series(h["train_acc"],  max_len) for h in all_histories], axis=0)
avg_val_acc    = np.mean([pad_series(h["val_acc"],    max_len) for h in all_histories], axis=0)
avg_train_loss = np.mean([pad_series(h["train_loss"], max_len) for h in all_histories], axis=0)
avg_val_loss   = np.mean([pad_series(h["val_loss"],   max_len) for h in all_histories], axis=0)
avg_val_auc    = np.mean([pad_series(h["val_auc"],    max_len) for h in all_histories], axis=0)
```

```
epochs = np.arange(1, max_len + 1)


early_stop_epoch = int(np.argmax(avg_val_auc) + 1)

# İlgili nokta değerleri
early_acc_y  = avg_val_acc[early_stop_epoch - 1]
early_loss_y = avg_val_loss[early_stop_epoch - 1]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# ---- Accuracy grafiği ----
ax1.plot(epochs, avg_train_acc, label="Train Accuracy", linewidth=2)
ax1.plot(epochs, avg_val_acc, label="Validation Accuracy", linewidth=2)

ax1.axvline(
    x=early_stop_epoch,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Early Stop Epoch = {early_stop_epoch}"
)

ax1.scatter(
    early_stop_epoch,
    early_acc_y,
    color="red",
    s=80,
    zorder=5
)

ax1.annotate(
    f"Early Stop Epoch = {early_stop_epoch}",
    xy=(early_stop_epoch, early_acc_y),
    xytext=(early_stop_epoch + 1, early_acc_y + 0.01),
    color="red",
    fontsize=10,
    arrowprops=dict(arrowstyle="->", color="red", lw=1.5)
)

ax1.set_title("GCN Average Classification Accuracy")
ax1.set_xlabel("Epoch")
ax1.set_ylabel("Accuracy")
ax1.legend()
ax1.grid(True, alpha=0.3)

# ---- Loss grafiği ----
ax2.plot(epochs, avg_train_loss, label="Train Loss", linewidth=2)
ax2.plot(epochs, avg_val_loss, label="Validation Loss", linewidth=2)

ax2.axvline(
    x=early_stop_epoch,
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"Early Stop Epoch = {early_stop_epoch}"
)

ax2.scatter(
    early_stop_epoch,
    early_loss_y,
    color="red",
    s=80,
    zorder=5
)

ax2.annotate(
    f"Early Stop Epoch = {early_stop_epoch}",
    xy=(early_stop_epoch, early_loss_y),
    xytext=(early_stop_epoch + 1, early_loss_y + (max(avg_val_loss) - min(avg_val_loss)) * 0.08),
    color="red",
    fontsize=10,
    arrowprops=dict(arrowstyle="->", color="red", lw=1.5)
)

ax2.set_title("GCN Average Loss")
ax2.set_xlabel("Epoch")
ax2.set_ylabel("Loss")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Early Stop Epoch = {early_stop_epoch}")
print("Fold bazlı epochs_ran:", results_df['epochs_ran'].tolist())
print("Ortalama epochs_ran:", results_df['epochs_ran'].mean())
```

Gizli çıkışı göster